

Raporty 2.0 - dashboard SOC Logsign z MySQL

Raporty 2.0 to kontenerowa aplikacja PHP/Nginx/MySQL do importowania raportów HTML z systemu Logsign, ich archiwizacji oraz zapisu znormalizowanych danych do bazy MySQL. Warstwa wizualna dashboardów raportów została zachowana w stylu wersji 1.x, natomiast statystyki zostały rozbudowane o odczyt z MySQL, dodatkowe zestawienia TOP oraz przyciski Pokaż pozostałe przy tabelach.

Najważniejsze zmiany w wersji 2.0

- zapis importowanych raportów do MySQL,
- zachowanie plików HTML w katalogu `src/dane/YYYY-MM-DD` jako archiwum,
- konfiguracja połączenia z bazą przez plik `.env`,
- automatyczne tworzenie schematu bazy przy starcie/importach,
- rozbudowany panel `stats.php` oparty o MySQL,
- tabele TOP z przyciskiem **Pokaż pozostałe**,
- port aplikacji ustawiony na `8094:80`,
- port MySQL ustawiony na `3309:3306`,
- kontenery nazwane prefiksem `raport2-\*`.

Architektura

```
ZIP z raportami Logsign
|
v
src/upload.php
|
+-- zapis HTML do src/dane/YYYY-MM-DD/
|
+-- parser PHP według typu raportu
|
v
MySQL: reports + report_events + import_logs
|
+-- index.php: obecny widok raportów z archiwum HTML
|
+-- stats.php: nowe statystyki z MySQL
```

Aplikacja działa w modelu hybrydowym: HTML-e zostają na dysku, natomiast dane analityczne trafiają do MySQL. Dzięki temu obecne widoki pozostają kompatybilne, a statystyki są szybsze i bardziej rozbudowane.

Wymagania

- Docker,
- Docker Compose,
- wolne porty `8094` oraz `3309`.

Uruchomienie

1. Rozpakuj paczkę projektu.
2. Wejdź do katalogu projektu:

```
cd raporty-2.0
```

3. Uruchom kontenery:

```
docker compose up -d --build
```

4. Otwórz aplikację:

```
http://localhost:8094
```

5. Panel statystyk MySQL:

```
http://localhost:8094/stats.php
```

Konfiguracja `.env`

Plik `.env` znajduje się w katalogu głównym projektu:

```
APP_NAME=Raporty 2.0  
APP_PORT=8094
```

```
MYSQL_HOST=db  
MYSQL_PORT=3306  
MYSQL_DATABASE=raporty_db  
MYSQL_ROOT_PASSWORD=root_password  
MYSQL_USER=raport_user  
MYSQL_PASSWORD=raport_pass
```

```
MYSQL_EXTERNAL_PORT=3309
```

Ważne: aplikacja PHP łączy się z bazą po nazwie usługi Docker Compose, czyli `db:3306`. Narzędzia zewnętrzne, np. DBeaver, łączą się z hosta przez `127.0.0.1:3309`.

Dane do DBeaver

```
Host: 127.0.0.1  
Port: 3309  
Database: raporty_db  
User: raport_user  
Password: raport_pass
```

Dla sterownika MySQL 8 w DBeaver można dodać właściwości:

```
allowPublicKeyRetrieval=true  
useSSL=false
```

Kontenery

Projekt używa następujących kontenerów:

raport2-web - Nginx, port aplikacji 8094
raport2-app - PHP-FPM 8.2 z rozszerzeniami ZIP i PDO MySQL
raport2-db - MySQL 8.0, port zewnętrzny 3309

Struktura projektu

```
raporty-2.0/  
■■■ .env  
■■■ docker-compose.yml  
■■■ Dockerfile  
■■■ nginx.conf  
■■■ upload.ini  
■■■ database/  
■ ■■■ schema.sql  
■■■ src/  
    ■■■ index.php  
    ■■■ upload.php  
    ■■■ stats.php  
    ■■■ config/  
    ■ ■■■ database.php  
    ■■■ db/  
    ■ ■■■ importer.php  
    ■ ■■■ schema.sql  
    ■■■ parsers/  
    ■ ■■■ parserzy raportów Logsign  
    ■■■ inc/  
    ■ ■■■ widoki dashboardów  
    ■■■ dane/  
    ■ ■■■ archiwum raportów HTML  
    ■■■ temp/  
        ■■■ katalog roboczy uploadu
```

Schemat bazy danych

`reports`

Tabela przechowuje jeden rekord na każdy zaimportowany raport HTML.

Najważniejsze kolumny:

- `report\_date` - data raportu,
- `report\_type` - wykryty typ raportu,
- `original\_filename` - nazwa oryginalna,
- `stored\_filename` - nazwa po normalizacji,
- `file\_path` - ścieżka do pliku HTML,
- `checksum` - SHA-256 do wykrywania duplikatów,
- `events\_total` - suma zdarzeń,
- `parser\_status` - status parsowania.

`report\_events`

Tabela przechowuje znormalizowane zdarzenia lub agregaty odczytane z parserów.

Obsługiwane pola obejmują m.in.:

- `source\_ip`,
- `source\_host`,
- `source\_user`,
- `source\_country`,
- `destination\_ip`,
- `destination\_port`,
- `destination\_country`,
- `protocol\_name`,
- `service\_name`,
- `application\_name`,
- `application\_category`,
- `event\_subtype`,
- `bytes\_rx`, `bytes\_tx`, `bytes\_total`,
- `raw\_payload` z oryginalnym wynikiem parsera w JSON.

`import\_logs`

Tabela pomocnicza na logi importów i błędów.

Obsługiwane raporty

Aplikacja zachowuje obsługę parserów z wersji 1.x:

- transfer hostów,
- hosty wewnętrzne skanujące porty,
- hosty zewnętrzne skanujące porty,
- błędne logowania według hostów,
- błędne logowania według użytkowników,
- odrzucone połączenia z hostów wewnętrznych,
- odrzucone połączenia z hostów zewnętrznych,
- połączenia wychodzące na niestandardowe porty.

Przepływ importu

1. Użytkownik wgrywa ZIP przez dashboard.
2. `upload.php` rozpakowuje paczkę, również ZIP-y zagnieżdżone.
3. Pliki HTML trafiają do `src/dane/YYYY-MM-DD/`.
4. Importer wylicza checksum SHA-256.
5. Jeżeli raport nie istnieje w bazie, tworzony jest rekord w `reports`.
6. Wykrywany jest typ raportu.
7. Uruchamiany jest odpowiedni parser.
8. Wynik parsera jest zapisywany do `report\_events`.
9. Panel statystyk korzysta już z MySQL.

Statystyki 2.0

Nowy panel `stats.php` pokazuje:

- liczbę raportów,
- sumę zdarzeń,
- liczbę unikalnych IP źródłowych,
- liczbę hostów,
- łączny transfer,
- trend dzienny zdarzeń,
- TOP typów raportów,
- TOP Source.IP,
- TOP Source.HostName,
- TOP Source.UserName,
- TOP Source.Country,
- TOP Destination.IP,
- TOP Destination.Port,
- TOP Destination.Country,
- TOP Protocol.Name,
- TOP Service.Name,
- TOP Application.Name,
- TOP Application.Category,
- TOP EventMap.SubType,
- TOP godzin,
- TOP transferu.

Przy tabelach z większą liczbą wyników dostępny jest przycisk Pokaż pozostałe.

Komendy administracyjne

Status kontenerów:

```
docker compose ps
```

Logi aplikacji:

```
docker compose logs app
```

Logi bazy:

```
docker compose logs db
```

Wejście do MySQL z kontenera:

```
docker exec -it raport2-db mysql -uraport_user -praport_pass raporty_db
```

Test tabel:

```
SHOW TABLES;
SELECT COUNT(*) FROM reports;
SELECT COUNT(*) FROM report_events;
```

Zatrzymanie:

```
docker compose down
```

Zatrzymanie z usunięciem danych bazy:

```
docker compose down -v
```

Bezpieczeństwo

- Produkcyjnie należy zmienić hasła w `.env`.
- Katalog `src/dane/` zawiera importowane raporty i może zawierać dane wrażliwe.
- Katalog `src/temp/` jest roboczy i powinien być czyszczony po imporcie.
- W środowisku produkcyjnym warto dodać autoryzację użytkowników, HTTPS oraz ograniczenie dostępu do panelu.

Uwagi developerskie

- Obecne widoki dashboardów w `src/inc/` zostały zachowane wizualnie.
- `index.php` nadal potrafi czytać raporty z archiwum HTML.
- `stats.php` jest już oparty o MySQL.
- Importer nie blokuje archiwizacji pliku, gdy zapis do MySQL zwróci błąd - błąd trafia do logów.
- Duplikaty raportów są wykrywane po checksum SHA-256.

Szybki scenariusz testowy

1. Uruchom projekt:

```
docker compose up -d --build
```

2. Wejdź na:

<http://localhost:8094>

3. Wgraj ZIP z raportami HTML.

4. Otwórz:

<http://localhost:8094/stats.php>

5. Sprawdź w MySQL:

```
docker exec -it raport2-db mysql -uraport_user -praport_pass raporty_db -e "SELECT COUNT(*)
FROM reports; SELECT COUNT(*) FROM report_events;"
```